



Instituto Politécnico Nacional



**UPIITA (Unidad Profesional
Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas)**



Base de Datos Distribuidos

Profesor: De la cruz Sosa Carlos

Asignación: Tarea 3

Alumno:

-Gutiérrez Flores José Julián

-Muñoz Reyes Citlali

Tarea 3

Título: Diseño de propuesta de fragmentación

Instrucciones:

1. Seleccionar uno de los 4 ejercicios compartidos sobre diseño de fragmentación o proponer uno considerando la BD Adventureworks.



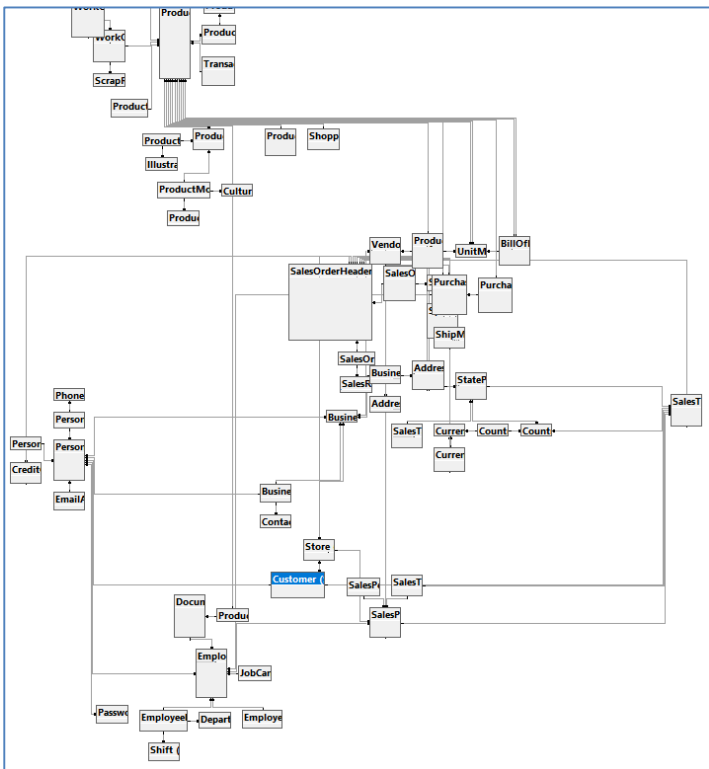
Ejercicio elegido:

Ejercicio 3: Distribuir la cartera de clientes en los 5 nodos geográficos para que cada región administre y consulte de forma autónoma los datos de sus propios compradores. Tabla Sales.Customer, columna CustomerId (rangos).

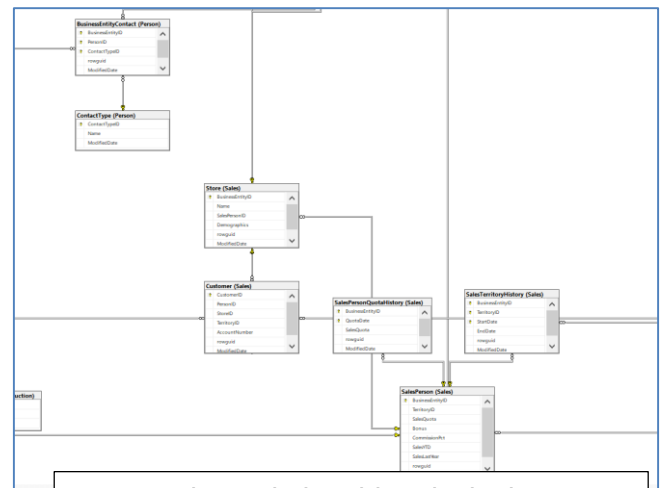
2. Generar el segmento de grafo relacional relacionado con el ejercicio seleccionado. Considere las tablas que se derivan de la tabla propietaria de su propuesta de diseño.

Tabla propietaria: Sales.Customer columna selector: columna CustomerId

Para determinar que tablas aparecerán en el segmento de grafo relacional, debemos de ver las relaciones, así como las llaves de la tabla que tomaremos como Tabla Propietaria



Visualización del diagrama completo de la base de datos

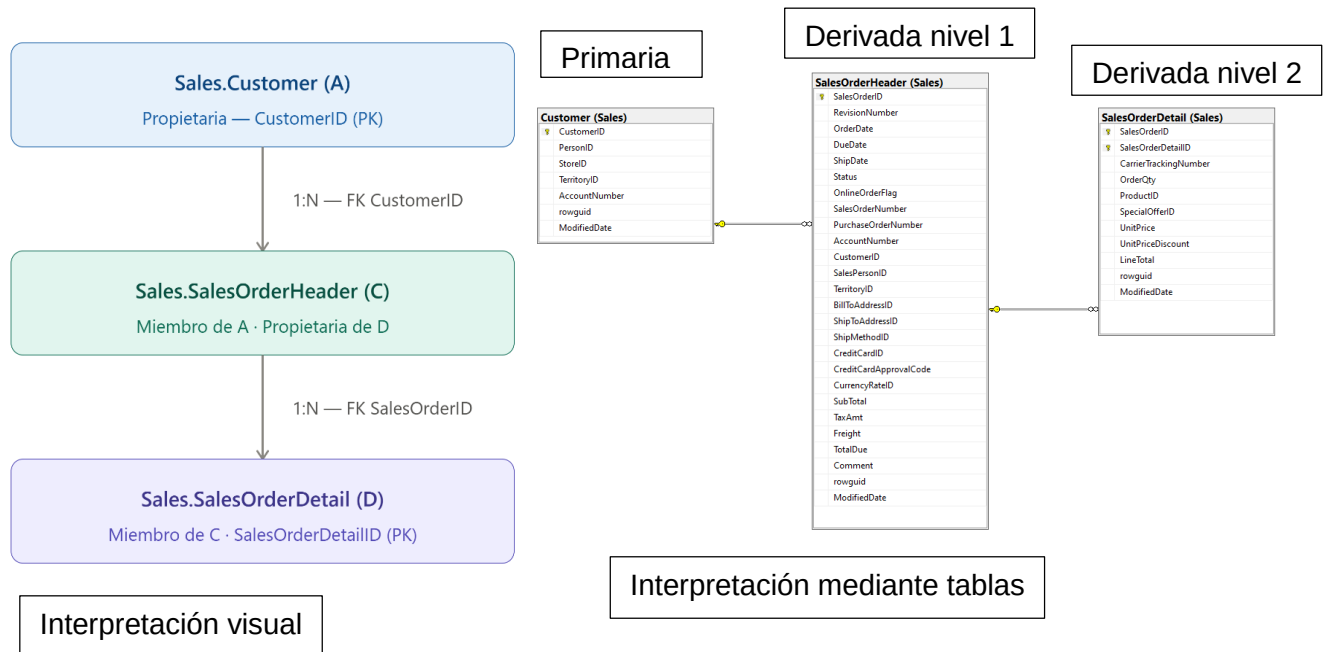


Acercamiento de la tabla principal para ver la relación

- Sales.Customer
- Columnas
- Claves
 - PK_Customer_CustomerID
 - FK_Customer_Person_PersonID
 - FK_Customer_SalesTerritory_TerritoryID
 - FK_Customer_Store_StoreID
- Restricciones
- Desencadenadores
- Índices
- Estadísticas

Visualización de las llaves que parte de la tabla principal o inicial.

Tras un análisis obtenemos las tablas derivadas, que son las que reciben como llave foránea la llave primaria de la tabla propietaria



3. Aplicar el algoritmo COM_MIN para generar los posibles fragmentos primarios de su diseño.

Para este ejercicio fragmentaremos horizontalmente la tabla propietaria Sales.Customer en 5 regiones (norte, sur, este, oeste, centro), para ello se aplica el algoritmo COM_MIN sobre el atributo de fragmentación CustomerID, con el fin de obtener el conjunto de predicados mínimo y completo (Pr') requeridos, según la teoría de Tamer Ozsu.

Predicados simples candidatos:

Dado que se buscan 5 regiones, basta con definir 4 puntos de corte sobre el dominio de CustomerID. Los valores de corte (R1, R2, R3, R4) se determinan a partir del dominio real de la tabla, obtenido mediante:

```
USE AdventureWorks2022;
GO
SELECT MIN(CustomerID) AS MinimoID, MAX(CustomerID) AS MaximoID, COUNT(*) AS Total
FROM Sales.Customer;
```

	MinimoID	MaximoID	Total
1	1	30118	19820

Retomando esta función de calcular MinimoID y MaximoID, podemos integrarla a una función de ventana para poderla ocupar junto con la función NTILE (Distribuye las filas de una partición ordenada en un número especificado de grupos) y que nos muestre los puntos de cortes.

```
WITH VentasOrdenadas AS (
    SELECT
        CustomerID,
        NTILE(5) OVER (ORDER BY CustomerID) AS NumeroNode
    FROM Sales.SalesOrderHeader
)
SELECT
    NumeroNode,
    MIN(CustomerID) AS MinCustomerID,
    MAX(CustomerID) AS MaxCustomerID,
    COUNT(*) AS TotalPedidos
FROM VentasOrdenadas
GROUP BY NumeroNode
ORDER BY NumeroNode;
```

R1= 13,629

R2= 17,245

R3= 21,921

R4= 27,239

El conjunto de predicados simples candidatos (Pr) queda definido como:

p1: CustomerID <=13,629

p2: CustomerID <= 17,245

p3: CustomerID <= 21,921

p4: CustomerID <= 27,239

Resultados		Mensajes		
	NumeroNodo	MinCustomerID	MaxCustomerID	TotalPedidos
1	1	11000	13629	6293
2	2	13629	17245	6293
3	3	17245	21921	6293
4	4	21922	27239	6293
5	5	27240	30118	6293

Paso	Predicado evaluado	Justificación de relevancia	Acción
1	p1	Separa la región norte del resto de las regiones; Q1–Q3 acceden de forma distinta según esta región	Se agrega a Pr'
2	p2	Particiona el fragmento $\neg p1$ en región sur vs. el resto	Se agrega a Pr'
3	p3	Particiona el fragmento $\neg p1 \wedge \neg p2$ en región este vs. el resto	Se agrega a Pr'
4	p4	Particiona el fragmento $\neg p1 \wedge \neg p2 \wedge \neg p3$ en región oeste vs. centro	Se agrega a Pr'

Resultado: Pr' = {p1, p2, p3, p4}

🌈 Mini términos de predicados -> combinación de atributos simples

La combinación de Pr' mediante conjunción (AND) produce los siguientes mini términos de predicados que definen los fragmentos primarios:

Mínterm	Condición	Región
m1	CustomerID $\leq$ R1	Norte
m2	R1 < CustomerID $\leq$ R2	Sur
m3	R2 < CustomerID $\leq$ R3	Este
m4	R3 < CustomerID $\leq$ R4	Oeste
m5	CustomerID > R4	Centro

Con lo anterior podemos acreditar las propiedades de COM_MIN como la completitud y minimalidad, puesto que la unión de los 5 mini términos de predicados cubre íntegramente el dominio de CustomerID y ningún predicado de Pr' es redundante.

4. Generar los fragmentos derivados a partir de la operación SEMI-JOIN.

La fragmentación horizontal derivada se utiliza para propagar la fragmentación de la tabla propietaria hacia sus tablas dependientes y mediante la operación relacional de semi-join, utilizando las relaciones de llave foránea. Dados los 5 fragmentos primarios, la fragmentación derivada se realiza en dos niveles:

Nivel 1

La tabla Sales.SalesOrderHeader se relaciona con Sales.Customer mediante la llave foránea CustomerID. Los fragmentos derivados H1, H2, H3, H4, H5 se obtienen filtrando las órdenes de compra pertenecientes a los clientes de cada región.

H1 norte

```
SELECT h.* FROM Sales.SalesOrderHeader h
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID <= 13629;
```

H2

```
SELECT h.*
FROM Sales.SalesOrderHeader h
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 13629 AND c.CustomerID <= 17245;
```

H3

```
SELECT h.*
FROM Sales.SalesOrderHeader h
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 17245 AND c.CustomerID <= 21921;
```

H4

```
SELECT h.*
FROM Sales.SalesOrderHeader h
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 21921 AND c.CustomerID <= 27239;
```

H5

```
SELECT h.*
FROM Sales.SalesOrderHeader h
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 27239;
```

Nivel 2

La tabla Sales.SalesOrderDetail no tiene una relación directa con Sales.Customer, pero se relaciona con Sales.SalesOrderHeader a través de la llave foránea SalesOrderID. Para mantener la consistencia en el nodo regional correspondiente, los fragmentos D1, D2, D3, D4, D5 se obtienen aplicando la operación semi-join sobre los fragmentos del nivel anterior.

D1

```
SELECT d.*
FROM Sales.SalesOrderDetail d
JOIN Sales.SalesOrderHeader h ON d.SalesOrderID =
h.SalesOrderID
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID <=13629;
```

D2

```
SELECT d.*
FROM Sales.SalesOrderDetail d
JOIN Sales.SalesOrderHeader h ON d.SalesOrderID =
h.SalesOrderID
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 13629 AND c.CustomerID <=17245;
```

```

D3
SELECT d.*
FROM Sales.SalesOrderDetail d
JOIN Sales.SalesOrderHeader h ON d.SalesOrderID =
h.SalesOrderID
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 17245 AND c.CustomerID <= 21921;

D4
SELECT d.*
FROM Sales.SalesOrderDetail d
JOIN Sales.SalesOrderHeader h ON d.SalesOrderID =
h.SalesOrderID
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 21921 AND c.CustomerID <= 27239;

D5
SELECT d.*
FROM Sales.SalesOrderDetail d
JOIN Sales.SalesOrderHeader h ON d.SalesOrderID =
h.SalesOrderID
JOIN Sales.Customer c ON h.CustomerID = c.CustomerID
WHERE c.CustomerID > 27239;

```

5. Validar el diseño de la fragmentación para verificar que cumplan las 3 reglas de la fragmentación según la teoría de Tamer Ozsü (libro de texto utilizado en el curso).

Toda fragmentación primaria o derivada debe cumplir tres reglas de corrección, esto para garantizar que la base de datos distribuida sea equivalente.

Regla de completitud

Un conjunto de fragmentos es completo si todo elemento (tupla) de la relación original está contenido en al menos uno de los fragmentos.

Validación:

- ❖ *Sales.Customer*: los minis terminos m1 <-> m5 obtenidos en COM_MIN cubren la totalidad del dominio de CustomerID, por lo que todo cliente queda incluido en exactamente uno de los fragmentos F1 <-> F5.
- ❖ *Sales.SalesOrderHeader*: dado que CustomerID es una llave foránea **obligatoria** hacia Customer, todo pedido pertenece necesariamente a algún cliente, y por lo tanto a alguna de las regiones F1 <-> F5. En consecuencia, la unión de H1 <-> H5 contiene todos los pedidos de la tabla original.
- ❖ *Sales.SalesOrderDetail*: SalesOrderID es una FK obligatoria hacia SalesOrderHeader, por lo que todo detalle de venta pertenece a algún pedido ya clasificado en H1<-> H5. La unión de D1 <-> D5 contiene todos los detalles de la tabla original.

Regla de reconstrucción

Debe existir un operador relacional que permita reconstruir la relación original a partir de sus fragmentos. En fragmentación horizontal, este operador es la unión.

Validación:

Customer = F1 $\cup$ F2 $\cup$ F3 $\cup$ F4 $\cup$ F5

Sales.SalesOrderHeader = H1 $\cup$ H2 $\cup$ H3 $\cup$ H4 $\cup$ H5

Sales.SalesOrderDetail = D1 $\cup$ D2 $\cup$ D3 $\cup$ D4 $\cup$ D5

Como cada fragmento conserva el mismo esquema (mismas columnas) que su tabla de origen, la operación de unión es válida y reconstruye exactamente la relación original, sin pérdida ni duplicidad de información.

Regla de disyunción

Los fragmentos deben ser mutuamente excluyentes: si una tupla aparece en un fragmento, no debe aparecer en ningún otro.

Sales.Customer: los predicados p1 $\leftrightarrow$ p4 usados en COM_MIN son rangos estrictamente excluyentes entre sí, debido a que cada mini termino define un intervalo semiabierto que no se traslapa con los demás, por lo que $F_i \cap F_j = \emptyset$ para todo $i \neq j$.

Sales.SalesOrderHeader: cada pedido tiene un único valor de CustomerID (relación N:1, no multivaluada), por lo que solo puede satisfacer el semi-join con un único fragmento F_i . Esto garantiza $H_i \cap H_j = \emptyset$ para todo $i \neq j$.

Sales.SalesOrderDetail: cada detalle tiene un único SalesOrderID, por lo que solo puede pertenecer a un único fragmento H_i , garantizando $D_i \cap D_j = \emptyset$ para todo $i \neq j$.

6. Concluir acerca del diseño propuesto.

-El diseño de fragmentación propuesto para AdventureWorks2022, distribuye la cartera de clientes y su actividad transaccional en 5 nodos regionales, mediante una combinación de fragmentación horizontal primaria en la tabla propietaria y fragmentación horizontal derivada en cascada sobre sus dos tablas dependientes. Esto al elegir al atributo CustomerID como base, permitiendo que se pueda obtener un conjunto de predicados mínimo y completo, del cual se derivaron 5 fragmentos primarios balanceados.

-La técnica de semi-join permitió propagar la misma partición geográfica hacia SalesOrderHeader y, en un segundo nivel, hacia SalesOrderDetail, sin necesidad de definir predicados independientes sobre estas tablas; cumpliendo así al final con las tres reglas de corrección de Tamer Ozsu, en sus tres niveles de la jerarquía; garantizando que ningún cliente, pedido o detalle de venta quede fuera de su fragmento correspondiente, ni duplicado entre dos fragmentos distintos.